numpy_info.ipynb

Подключение библиотек

установка

Для установки внешних библиотек, которые не входят в стандартную библиотеку Python, используйте менеджер пакетов pip. Например в jupyter notebook или в google colab
```
!pip install pandas
```

В консоле:
```
pip install pandas
```

```
!pip install pandas
```

import

В Python подключение библиотек осуществляется с помощью команды **import**. Существуют разные способы импорта библиотек в зависимости от ваших нужд:

Импорт всей библиотеки:
```
import math
```

В этом случае вы можете использовать функции и переменные из библиотеки, обращаясь к ним через имя библиотеки:
```
print(math.sqrt(16))
```

Импорт конкретных функций или классов (не всей библиотеки, а то что нужно): Это позволяет использовать импортированные элементы напрямую, без указания имени библиотеки:
```
from math import pi, sqrt
print(sqrt(16))
print(pi)
```

Импорт с псевдонимом (алиас):
```
import math as mt
print(mt.sqrt(16))
```

Импорт всех элементов библиотеки (не рекомендуется)
```
from math import *
```

```

```

Стандартные библиотеки Python

Стандартные библиотеки — это модули, которые не требуют дополнительной установки и доступны сразу после установки Python.

os — Взаимодействие с операционной системой

Основные функции:

- os.getcwd(): получить текущую рабочую директорию

- os.listdir(path): список файлов в директории
- os.mkdir(path): создать директорию
- os.remove(path): удалить файл
- os.path.join(): объединить пути

```
import os
path = 'sample_data'
print(os.listdir(path))

#os.mkdir('qwe')
# # Удалить директорию
# if os.path.exists('test_dir'):
#     os.rmdir('test_dir')
#     print("Директория удалена")
```

```

```

sys — Системные параметры и функции

- sys.version: версия Python
- sys.argv: аргументы командной строки
- sys.exit(): выход из программы
- sys.stdout: стандартный вывод

```
import sys

print("Версия Python:", sys.version)
```

math - содержит математические функции и константы.

Основные функции:

- math.sqrt(x): квадратный корень
- math.sin(x), math.cos(x): тригонометрия
- math.pi, math.e: константы
- math.log(x): логарифм
- math.factorial: факториал

```
import math

print("Квадратный корень из 25:", math.sqrt(25))
print("Синус 90 градусов (в радианах):", math.sin(math.pi / 2))
print("Число Пи:", math.pi)
print("Логарифм 100 по основанию 10:", math.log10(100))
print("факториал 10:", math.factorial(10))
```

random - Генерация случайных чисел

Модуль random используется для генерации случайных чисел, выбора элементов и т.д.

Основные функции:

- random.random(): случайное число от 0 до 1
- random.randint(a, b): случайное целое число
- random.choice(seq): случайный элемент из последовательности
- random.shuffle(seq): перемешать последовательность

```
import random


fruits = ['яблоко', 'банан', 'апельсин']
print("Случайный фрукт:", random.choice(fruits))
random.shuffle(fruits)
fruits
```

```
import random

print("Случайное число от 0 до 1:", random.random())
print("Случайное целое от 1 до 10:", random.randint(1, 10))

# Случайный выбор
fruits = ['яблоко', 'банан', 'апельсин']
print("Случайный фрукт:", random.choice(fruits))

# Перемешивание списка
numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)
print("Перемешанный список:", numbers)
```

collections - Специализированные контейнеры

Модуль collections предоставляет альтернативы стандартным контейнерам с дополнительными возможностями.

Основные классы:

- collections.Counter: счетчик элементов
- collections.defaultdict: словарь с дефолтными значениями
- collections.namedtuple: кортеж с именованными полями

```
from collections import Counter, defaultdict, namedtuple

words = ['apple', 'banana', 'apple', 'orange', 'banana']
count = Counter(words)
print("Счетчик слов:", count)
```

itertools - Итераторы и комбинаторика

Модуль itertools предоставляет функции для создания итераторов.

Основные функции:

- itertools.permutations(iterable, r): перестановки
- itertools.combinations(iterable, r): комбинации

- itertools.cycle(iterable): бесконечный цикл

```
import itertools

# Перестановки
perms = list(itertools.permutations(['A', 'B', 'C'], 2))
print("Перестановки:", perms)

# Комбинации
combs = list(itertools.combinations(['A', 'B', 'C', 'D'], 2))
print("Комбинации:", combs)

# Цикл
cycle = itertools.cycle(['A', 'B', 'C'])
print("Цикл:", [next(cycle) for _ in range(5)])
```

И т.д

- time - обработка времени
- datetime - обработка времени и даты
- re - Регулярные выражения

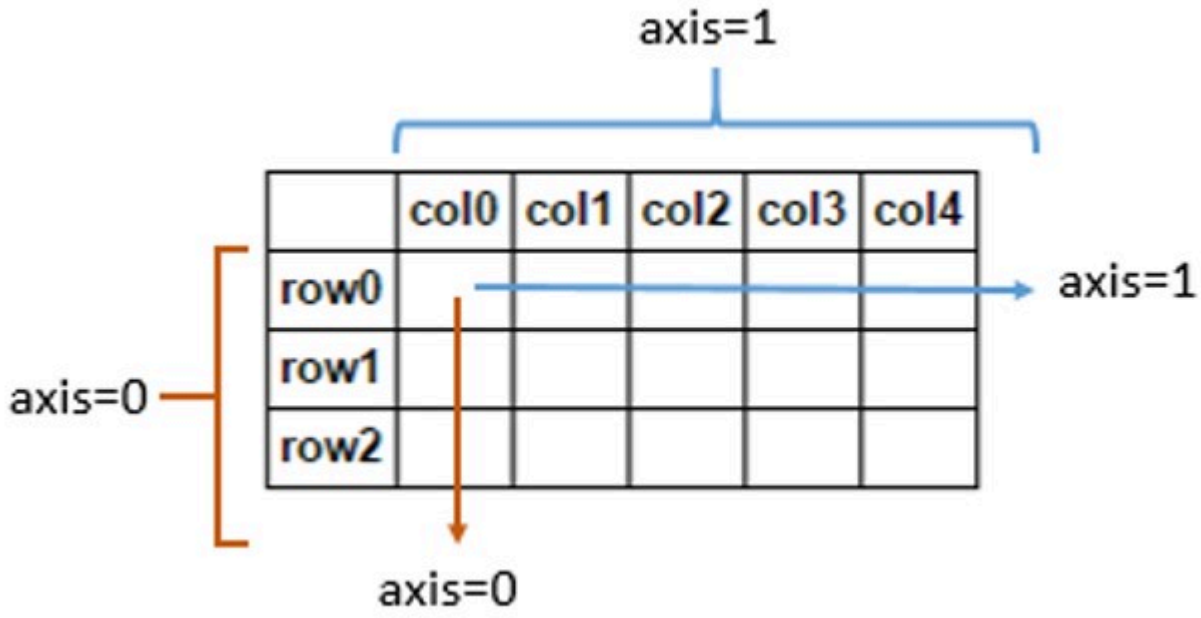
Полный список можно найти в официальной документации Python https://docs.python.org/3/library/ (https://docs.python.org/3/library/)

```
import time
start = time.time()

count = 0
while True:
  if count > 100:
    break
  count += 1
  time.sleep(0.05)
end = time.time() - start
end
```

Numpy

```
import numpy as np
```



Производительность numpy

Посмотрим на простой пример — сумма первых 10^8 чисел.

```
%%time

sum_value = 0
for i in range(10 ** 8):
    sum_value += i
print(sum_value)
```

```
%%time

sum_value = sum(range(10 ** 8))
print(sum_value)
```

```
%%time

sum_value = np.arange(10 ** 8).sum()
print(sum_value)
```

Задание массива

```
l = [[1,2,3,44]]
np.array(l).T
```

1. Массив из списка

```
arr1 = np.array([1, 2, 3]) # Массив из списка
print(arr1)
```

```
arr1 = np.array([
    [1, 2, 3],
    [0, 3, 10]
    ]) # Массив из списка
print(arr1) # (2, 3) (3, ..)
```

```
arr1 @ arr1.T
```

2. Массив из заполненый нулями

```
arr2 = np.zeros((3, 3))  # Массив 2x3 заполненный нулями
#print(arr2, end='\n')
arr_e = np.identity(3)
print(arr_e)
```

3. Массив заполненный единицами

```
arr3 = np.ones((3, 2))   # Массив 3x2 заполненный единицами
print(arr3, end='\n')
```

```
# range(0, 100, 1)
```

4. Массив с шагом (аналог range). **Аргументы могут быть с плавающей точкой.**

```
# np.arange(0.7, 100.5, 0.1)
```

```
arr4 = np.arange(0, 10, 2)  # Массив от 0 до 10 с шагом 2
print(arr4, end='\n')
```

```
print(np.arange(0, 10, 2))
print(list(range(0, 10, 2)))
```

5. Последовательности чисел с постоянным шагом можно также создавать функцией linspace. Начало и конец диапазона включаются; последний аргумент — число точек.

```
arr5 = np.linspace(0, 1, 5)  # Массив из 5 чисел от 0 до 1
print(arr5)
```

```
# np.linspace(0, 100, 1000)
```

```

```

```
print(np.linspace(0, 1, 10))
```

Информация о массиве (тип и размеры)

```
arr1.shape
```

```
arr = np.array(
    [
    [1, 2, 3],
    [4, 5, 6],
    ]
    )
print("Shape:", arr.shape)  # Вывод: (2, 3)
print("Data type:", arr.dtype)  # Вывод: int64
print("Number of dimensions:", arr.ndim)  # Вывод: 2
print("Number of dimensions:", len(arr))  # Вывод: 2
print("Size:", arr.size)  # Вывод: 6
```

Операции с массивами

```
a = np.array([
    [1, 2, 3]
    ])

b = np.array([
    [1, 2, 3],
    [4, 5, 6]
    ])
```

```
a.shape, b.T.shape
```

```
(a @ b.T).shape
```

```
# Транспонирование
a.T
```

1. Операции с массивом и числами

```
# Сложение
result_add = a + 2  # [3, 4, 5]

# Вычитание
result_sub = a - 2  # [-1, 0, 1]

# Умножение
result_mul = a * 2  # [2, 4, 6]

# Деление
result_div = a / 2  # [0.5, 1.0, 1.5]
print(result_div)
```

2. Операции между массивами

```
# Элемент-wise сложение
result_add_arrays = a + a  # [2, 4, 6]

# Элемент-wise умножение (Адамарово)
result_mul_arrays = a * a  # [1, 4, 9]

# Элемент-wise деление
result_div_arrays = a / a  # [1, 1, 1]

# Матричное умножение
result_mul = a @ a.T  # [14]
```

```

```

```
arr2 = np.zeros((2, 3))

arr2 + [1, 2, 3]
```

```
arr2 + [[1], [2]]
```

3. Изменение формы массива

```
# Изменение формы
reshaped_b = b.reshape((3, 2))  # [[1, 2], [3, 4], [5, 6]]

# Уплощение (вытягивание в вектор) массива
flattened_b = b.flatten()  # [1, 2, 3, 4, 5, 6]

b.flatten().reshape((6,1))
```

```
np.arange(0, 25).reshape(5, 5)
```

4. Объединение и разделение массивов

```
# Объединение массивов
concatenated = np.concatenate((a, a))  # [1, 2, 3, 1, 2, 3]

# Разделение массивов
split_a = np.split(b, 2)  # [array([[1, 2, 3]]), array([[4, 5, 6]])]
```

```
np.hstack((a, a)) # гаризонтальное объединение (по столбцам)
```

```
np.vstack((a, a)) # вертикальное объединение (по строкам)
```

5. Индексация и срезы

Как в list

```
b[0]
```

```
b[0][0]
```

```
b[::-1]
```

Далее модификации индексов и срезов

```
element_2d = b[1, 0]  # Элемент в строке 1, столбце 0
```

```
b[1, 1], b
```

5.1 Логические операции

```
# Логическое сравнение
bool_array = b >= 2  # bool_array — бинарная маска

# Применение логических операций
filtered_a = b[bool_array]
```

```
bool_array
```

```
b[bool_array]
```

```
b, b[~(b <= 3)]
```

Broadcasting

Выше при арифметических операциях с массивами, например, при сложении и умножении, мы перемножали массивы одинаковой формы. В самом простом случае операндами были одномерные массивы одинаковой длины.

```
# Самый простой случай
a = np.array([1, 2, 3])
b = np.array([2, 2, 2])
print(a * b)
```

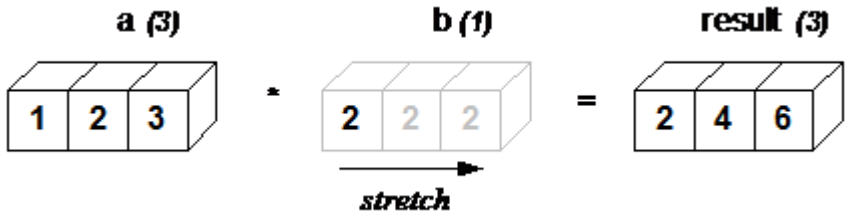
Произошло поэлементное умножение, все элементы массива a умножились на 2 . Но мы знаем, что это можно сделать проще, просто умножив массив на 2 .

```
# Умножение массива на число
print(a * 2)
```

На самом деле поведение будет аналогичным, если умножить одномерный массив на массив длины 1 .

```
# Умножение массивов разных длин
print(a * [2])
```

В этом случае работает так называемый *broadcasting*. Один массив "растягивается", чтобы повторить форму другого.



Такой же эффект работает и для многомерных массивов. Если по какому-то измерению размер у одного массива равен 1, а у другого — произвольный, то по этому измерению может произойти "рястяжение". Таким образом, массивы можно умножать друг на друга, если в измерениях, где они по размеру не совпадают, хотя бы у одного размер 1. Для других поэлементных операций правило аналогично.

Важно отметить, что размерности сопоставляются справа налево. Если их количество не совпадает, что массивы меньшей размерности сначала дополняются слева размерностями 1. Например, при сложении массива размера 4×3 с массивом размера 3 последний сначала преобразуется в массив размера 1×3.

```
a = np.array([[ 0,  0,  0],
              [10, 10, 10],
              [20, 20, 20],
              [30, 30, 30]])

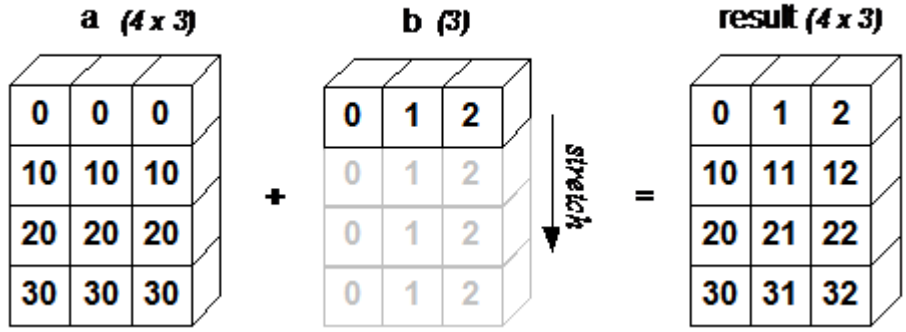
b = np.array([0, 1, 2])
```

```
a = np.array([[ 0,  0,  0],
              [10, 10, 10],
              [20, 20, 20],
              [30, 30, 30]])

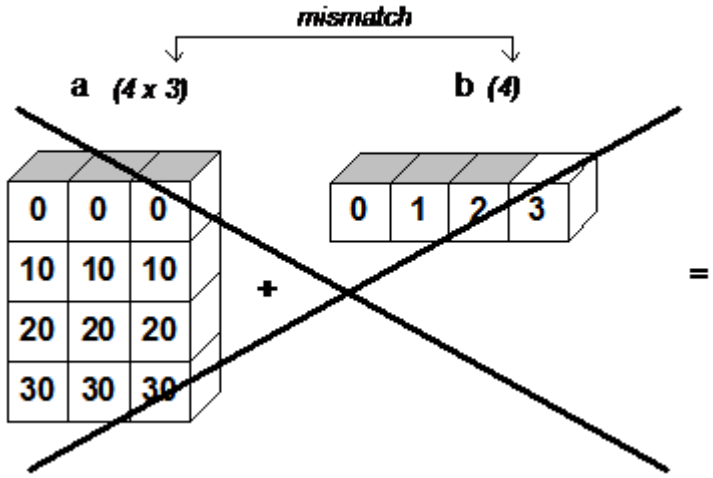
b = np.array([0, 1, 2])

print(a + b)
```

Схематично проведенную операцию можно визуализировать следующим образом.



Если неединичные размерности справа не будут совпадать, то выполнить операцию уже не получится. Например, как приведено на схеме ниже.




```
a = np.array([[ 0,  0,  0],
              [10, 10, 10],
              [20, 20, 20],
              [30, 30, 30]])

b = np.array([0, 1, 2, 3])

print(a + b)
```

Замечание*

Знать про broadcasting нужно, но пользоваться им надо с осторожностью. Многократное копирование массива при растяжении может привести к неэффективной работе программы по памяти.

Линейная алгебра

Решение уравнений

```
# Коэффициенты многочлена
coefficients = [1, -6, 11, -6]
# x^3 - 6x^2 + 11x - 6
# Нахождение корней
roots = np.roots(coefficients)
print("Корни многочлена:", roots)
```

```
a = np.array([[0, 1], [2, 3]])
```

Определитель матрицы (детерминант)
`np.linalg.det`

```
np.linalg.det(a) # детерминант
```

Обратная матрица
`np.linalg.inv`

```
# A * A(-1) = E
```

```
a1 = np.linalg.inv(a) # Обратная матрица
print(a1)
```

Решение линейной системы Ax=b
`np.linalg.solve(A, b)`

```
# Ax = b
# A*A^(-1)x= A^(-1)b
# x = A^(-1)b
```

```
b = np.array([0, 1])
print(a1 @ b)
```

```
u = np.linalg.solve(a, b)
print(u)
```

Собственные значения и собственные векторы: $au_i = \lambda_i u_i$. `l` — одномерный массив собственных значений λ_i, столбцы матрицы u — собственные векторы u_i.
`np.linalg.eig`

```
l, u = np.linalg.eig(a)
print(l)
```

```
print(u)
```

Функция diag от одномерного массива строит диагональную матрицу; от квадратной матрицы — возвращает одномерный массив её диагональных элементов.
`np.diag`

```
L = np.diag(l)
print(L)
#print(np.diag(L))
```

Все уравнения $au_i = \lambda_i u_i$ можно собрать в одно матричное уравнение $au = u\Lambda$, где Λ — диагональная матрица с собственными значениями λ_i по диагонали.

```
print(a @ u - u @ L)
```

Поэтому $u^{-1}au = \Lambda$.

```
print(np.linalg.inv(u) @ a @ u)
```

Интегрирование и дифференцирование

В библиотеке NumPy есть несколько функций, которые могут помочь вам с интегрированием и дифференцированием. Однако для более сложных задач часто используются другие библиотеки, такие как SciPy. Вот основные способы выполнения интегрирования и дифференцирования с использованием NumPy и SciPy.

1. Дифференцирование

```
np.gradient
def f(x):
    return x**2

# Создаем массив значений x
x = np.linspace(0, 10, 100)
y = f(x)

# Вычисляем производную
dy = np.gradient(y, x)
```

```
def f(x):
    return x**2

# Создаем массив значений x
x = np.linspace(-10, 10, 100)
y = f(x)

# Вычисляем производную
dy = np.gradient(y, x)
```

```
from matplotlib import pyplot as plt
```

```
plt.plot(x, y, label='f(x) = x^2')
plt.plot(x, dy, label="f'(x) = 2x", linestyle='--')
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.title('Дифференцирование с использованием NumPy')
plt.show()
```

2. Интегрирование

`np.trapz` – метод трапеция (численное вычисление интеграла)

```
# Вычисляем интеграл методом трапеций
integral = np.trapezoid(y, x)

print(f"Интеграл от x^2 от -10 до 10: {integral}")
```

3. Использование SciPy для интегрирования

```
from scipy.integrate import quad

# Интегрируем функцию от 0 до π
integral, error = quad(f, -10, 10)

print(f"Интеграл от sin(x) от 0 до π: {integral}, ошибка: {error}")
```

Статистика

Примечание: Модуль scipy.stats, который больше подходит под вероятностно-статистические задачи.

Основные статистические функции NumPy

NumPy предоставляет множество функций для выполнения статистических расчетов. Вот некоторые основные функции и примеры их использования.

```
data = np.array([1, 2, 3, 4, 5, 10, 241])
```

1. Среднее (mean):

```
mean = np.mean(data)
print(f"Среднее: {mean}")
```

2. Медиана (median):

```
median = np.median(data)
print(f"Медиана: {median}")
```

3. Стандартное отклонение (standard deviation):

```
std_dev = np.std(data)
print(f"Стандартное отклонение: {std_dev}")
```

4. Дисперсия (variance):

```
variance = np.var(data)
print(f"Дисперсия: {variance}")
```

5. Минимум и максимум (min, max):

```
minimum = np.min(data)
maximum = np.max(data)
print(f"Минимум: {minimum}, Максимум: {maximum}")
```

6. Квартиль (percentile):

```
q1 = np.percentile(data, 25)  # 25-й процентиль
q2 = np.percentile(data, 50)  # 50-й процентиль (медиана)
q3 = np.percentile(data, 75)  # 75-й процентиль
print(f"Квартиль Q1: {q1}, Q2: {q2}, Q3: {q3}")
```

7. Корреляция (correlation):

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([5, 4, 3, 2, 1])
correlation = np.corrcoef(x, y)[0, 1]
print(f"Корреляция: {correlation}")
```

Распределения и случайные числа

Случайные числа

Генерация случайных чисел:

```
np.random.rand(d0, d1, ..., dn) # Генерирует случайные числа из равномерного распределения на интервале [0, 1).

np.random.randn(d0, d1, ..., dn) # Генерирует случайные числа из стандартного нормального распределения (среднее 0 и стандартное отклонение 1).

np.random.randint(low, high=None, size=None, dtype=int) # Генерирует случайные целые числа в заданном диапазоне.
```

Распределения

1. Нормальное распределение

```
np.random.normal(loc=0.0, scale=1.0, size=None)
```

Генерирует случайные числа из нормального распределения с заданным средним (loc) и стандартным отклонением (scale).

```
# Генерация 1000 случайных чисел из нормального распределения
mu, sigma = 0, 1  # среднее и стандартное отклонение
data = np.random.normal(mu, sigma, 1000)

# Визуализация
plt.hist(data, bins=30, density=True, alpha=0.6, color='g')


# Плотность вероятности
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = np.exp(-0.5 * ((x - mu) / sigma) ** 2) / (sigma * np.sqrt(2 * np.pi))
plt.plot(x, p, 'k', linewidth=2)
plt.title('Нормальное распределение')
plt.show()
```

2. Распределение равномерное

```
np.random.uniform(low=0.0, high=1.0, size=None)
```

Генерирует случайные числа из равномерного распределения на заданном интервале.

```
# Генерация 1000 случайных чисел из равномерного распределения
low = 0
high = 1
data_uniform = np.random.uniform(0, 1, 1000) #

# Визуализация
# x = np.linspace(low, high, 1000)
# y = np.ones_like(x) / (high - low)  # Плотность вероятности равномерного распределения
# plt.plot(x, y, 'k', linewidth=2, label='Теоретическая кривая')

plt.hist(data_uniform, bins=30, density=True, alpha=0.6, color='b')
plt.title('Равномерное распределение')
plt.show()
```

3. Распределение Пуассона

```
np.random.poisson(lam=1.0, size=None)
```

Генерирует случайные числа из пуассоновского распределения с параметром lam (среднее значение)

```
# Генерация 1000 случайных чисел из распределения Пуассона
lambda_ = 5  # параметр
data_poisson = np.random.poisson(lambda_, 1000)

# Визуализация
# from scipy.stats import poisson
# x = np.arange(0, max(data_poisson) + 1)  # Значения, для которых будем вычислять плотность
# y = poisson.pmf(x, lambda_)  # Вероятности для распределения Пуассона
# plt.plot(x, y, 'k', linewidth=2, label='Теоретическая кривая')

plt.hist(data_poisson, bins=30, density=True, alpha=0.6, color='r')
plt.title('Распределение Пуассона')
plt.show()
```

4. Экспоненциальное распределение

```
np.random.exponential(scale=1.0, size=None):
```

Генерирует случайные числа из экспоненциального распределения с заданным масштабом

```
# Генерация 1000 случайных чисел из экспоненциального распределения
scale = 1.0  # параметр
data_exponential = np.random.exponential(scale, 1000)

# Визуализация
# x = np.linspace(0, max(data_exponential), 1000)  # Значения для плотности
# y = scale * np.exp(-scale * x)  # Плотность вероятности экспоненциального распределения
# plt.plot(x, y, 'k', linewidth=2, label='Теоретическая кривая')


plt.hist(data_exponential, bins=30, density=True, alpha=0.6, color='m')
plt.title('Экспоненциальное распределение')
plt.show()
```

5. Биномиальное распределение

```
np.random.binomial(n, p, size=None)
```

Генерирует случайные числа из биномиального распределения с n испытаниями и вероятностью успеха p.

```
# Генерация 1000 случайных чисел из биномиального распределения
n, p = 1, 0.3  # число испытаний и вероятность успеха
data_binomial = np.random.binomial(n, p, 1000)

# Визуализация
plt.hist(data_binomial, bins=30, density=True, alpha=0.6, color='c')
plt.title('Биномиальное распределение')
plt.show()
```